

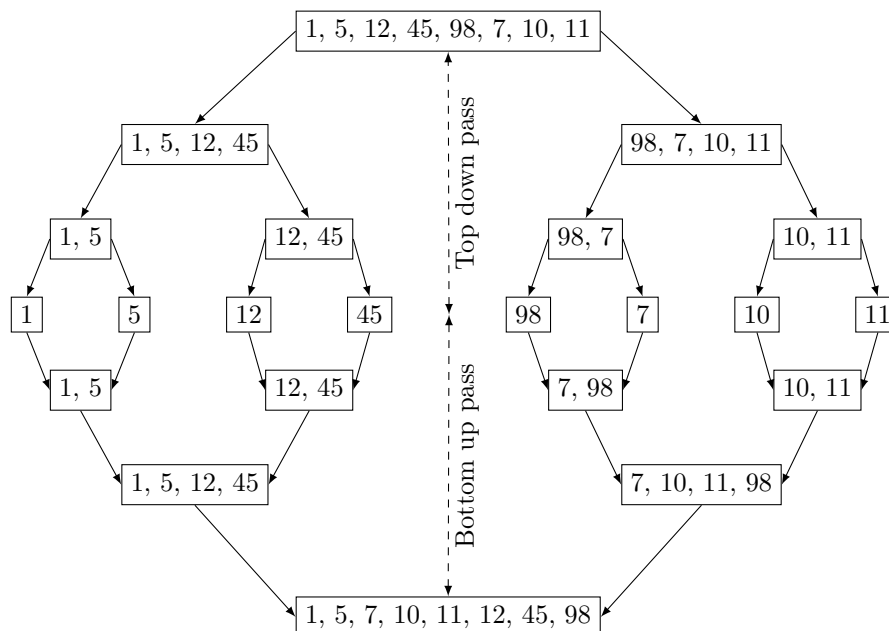
Merge sort

The core operation in merge sort is the merging process, which combines two sorted arrays into a single sorted array. To fully grasp merge sort, it is most effectively approached from a top-down perspective. The initial step involves recursively dividing the input array into two halves until we obtain subarrays of unit length. Specifically, for an input of size n , we generate...

- Two partitions of size $n/2$ each in first level,
- Four partition of size $n/4$ in second level,
- Eight partitions of size $n/8$ each in the third level, and so on,
- Until we get n partitions of size 1 each after $\log n$ partitions.

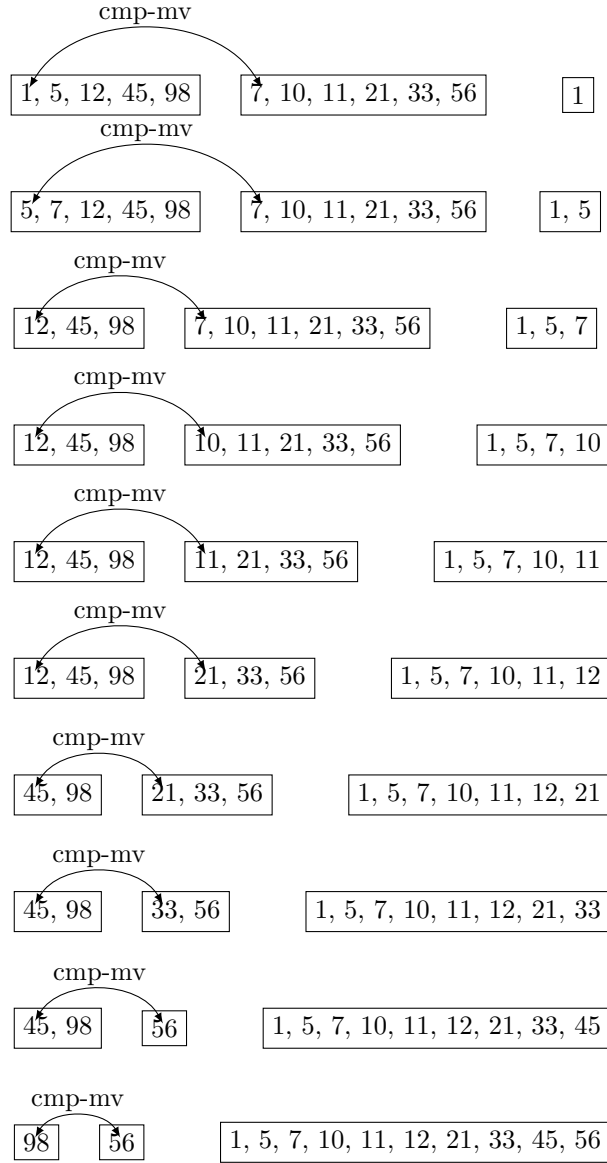
If the size of the array is not a power of 2, we can effectively pad it with the necessary number of dummy elements to expand its size to the nearest power of 2. This strategy ensures the process remains efficient, even when dealing with array sizes that aren't powers of 2.

To illustrate this concept more clearly, let's consider an example.



Following the top-down partitioning approach outlined earlier, we merge each pair of adjacent subarrays in a bottom-up manner to create sorted subarrays that are double the size. This merging process continues with adjacent pairs at each level until we get a single, fully sorted array. When we examine in-place merging, it essentially acts as a variation of insertion sort. In-place merging means integrating the two arrays without utilizing additional storage. In this method, one subarray takes on the role of donor while the other becomes the recipient; as a result, the donor subarray decreases in size while the recipient expands.

In practice, however, we use a separate array for the result or the output. The process involves comparing pairs of elements—one from a pair of sorted arrays and placing the smaller element into the result array. It then moves to the next element in the array from which the last element was taken. This compare-and-move operation continues until all elements from one of the subarrays have been added to the result. Finally, the remaining elements from the other array are added to the result to complete the output. The entire process is illustrated in the figure below with an example.



For implementation, we use two distinct cursors—one for each sorted input array. This approach allows us to compare the elements these cursors point to, and transfer the smaller element to the result array. When we move an element from one of the arrays into the result, we advance its cursor by one position to the right. In the accompanying figure, we purposefully omit explicit representations of the cursors, instead focusing on the starting positions of the arrays and elegantly masking the left segments that have already been incorporated into the output. So,

the figure enhances clarity of merging procedure. The complexity of merging process is $O(n)$ assuming n to be the combined size of two subarrays.

To prove the correctness of merge sort, we employ induction. The base case for our induction occurs when the input array consists of a single element. Since an array with one element is inherently sorted, we can confirm that the base case is valid.

Next, we assume that merge sort can correctly sort any array with a length less than n . When we apply merge sort to an array of size n , it will recursively call merge sort on two subarrays, each of size $n/2$. By the induction hypothesis, we can assert that the recursive application of merge sort on arrays of size $n/2$ will yield correctly sorted arrays.

Consequently, we conclude that both subarrays of size $n/2$ will be sorted after the recursive calls are completed. The overall correctness of merge sort thus hinges on the effectiveness of its merge operation, which combines the two sorted arrays of size $n/2$ each.

Now, let us analyze the computational complexity of merge sort. Merging two sorted arrays has a worst-case time complexity of $O(n)$, as it requires a single pass through each of the sorted arrays. We can denote the computational complexity as $T(n)$. The process of division leads us to the following recurrence relation:

$$T(n) = 2T(n/2) + O(n)$$

The solution to this recurrence equation yields a time complexity of $O(n \log n)$.